

Algorithmic Trading Scripts

The Complete Masterclass for MetaTrader 5

White Solutions

Published: September 2026

Copyright © 2026 White Solutions

All rights reserved.

This document is licensed for individual use.

Retail Price: \$450.00 USD

Contents

Foreword	v
1 Introduction to Algorithmic Finance	1
1.1 The Rise of Automation	1
1.2 System Architecture	2
2 MQL5 Fundamentals and Execution Flow	3
2.1 Event-Driven Architecture	3
2.2 Data Types and Memory Management	4
3 Mathematical Models and Indicators	7
3.1 Moving Averages	7
3.2 Relative Strength Index (RSI)	8
3.3 Position Sizing Models: The Kelly Criterion	8
4 Commercializing EAs: Security and Licensing	9
4.1 Implementing License Checks	9
5 Execution Logs and Strategy Tester	11
5.1 Analyzing the Logs	11

6	Complete Master EA Boilerplate	13
6.1	Advanced Trading Module: Part 1	13
6.1.1	Module 1 Architectural Analysis	15
6.2	Advanced Trading Module: Part 2	16
6.2.1	Module 2 Architectural Analysis	17
6.3	Advanced Trading Module: Part 3	18
6.3.1	Module 3 Architectural Analysis	19
6.4	Advanced Trading Module: Part 4	20
6.4.1	Module 4 Architectural Analysis	21
6.5	Advanced Trading Module: Part 5	22
6.5.1	Module 5 Architectural Analysis	23
6.6	Advanced Trading Module: Part 6	24
6.6.1	Module 6 Architectural Analysis	25
6.7	Advanced Trading Module: Part 7	26
6.7.1	Module 7 Architectural Analysis	27
6.8	Advanced Trading Module: Part 8	28
6.8.1	Module 8 Architectural Analysis	29
6.9	Advanced Trading Module: Part 9	30
6.9.1	Module 9 Architectural Analysis	31
6.10	Advanced Trading Module: Part 10	32
6.10.1	Module 10 Architectural Analysis . . .	33
6.11	Advanced Trading Module: Part 11	34
6.11.1	Module 11 Architectural Analysis . . .	35
6.12	Advanced Trading Module: Part 12	36
6.12.1	Module 12 Architectural Analysis . . .	37
6.13	Advanced Trading Module: Part 13	38
6.13.1	Module 13 Architectural Analysis . . .	39
6.14	Advanced Trading Module: Part 14	40

6.14.1	Module 14 Architectural Analysis . . .	41
6.15	Advanced Trading Module: Part 15	42
6.15.1	Module 15 Architectural Analysis . . .	43
6.16	Advanced Trading Module: Part 16	44
6.16.1	Module 16 Architectural Analysis . . .	45
6.17	Advanced Trading Module: Part 17	46
6.17.1	Module 17 Architectural Analysis . . .	47
6.18	Advanced Trading Module: Part 18	48
6.18.1	Module 18 Architectural Analysis . . .	49
6.19	Advanced Trading Module: Part 19	50
6.19.1	Module 19 Architectural Analysis . . .	51
7	Standard Library API Reference	53
7.1	OrderSend	53
7.2	iMA	54
7.3	iRSI	54
7.4	CopyBuffer	55
7.5	AccountInfoDouble	56
7.6	WebRequest	56
7.7	OrderSend	57
7.8	iMA	57
7.9	iRSI	58
7.10	CopyBuffer	59
7.11	AccountInfoDouble	59
7.12	WebRequest	60
7.13	OrderSend	60
7.14	iMA	61
7.15	iRSI	62

7.16	CopyBuffer	62
7.17	AccountInfoDouble	63
7.18	WebRequest	63
7.19	OrderSend	64
7.20	iMA	65
7.21	iRSI	65
7.22	CopyBuffer	66
7.23	AccountInfoDouble	66
7.24	WebRequest	67
7.25	OrderSend	68
7.26	iMA	68
7.27	iRSI	69
7.28	CopyBuffer	69
7.29	AccountInfoDouble	70
7.30	WebRequest	71
8	Conclusion and Post-Deployment Maintenance	73

Foreword

The financial markets have evolved. Discretionary trading, driven by human emotion and manual execution, is rapidly being replaced by quantitative, algorithmic solutions. At **White Solutions**, our mission is to empower developers, financial engineers, and retail traders with enterprise-grade trading architectures.

This premium guide is designed to take you from the fundamentals of algorithmic theory to the deployment of commercial-grade Expert Advisors (EAs) using MQL5 for MetaTrader 5. We will cover advanced mathematical modeling, strategy tester execution logs, and crucial commercialization steps, including how to implement licensing checks to protect your proprietary trading scripts.

Chapter 1

Introduction to Algorithmic Finance

Algorithmic trading uses mathematical models and automated systems to execute trades in financial markets. These algorithms analyze vast amounts of market data, identify trends, and execute orders at speeds impossible for human traders.

1.1 The Rise of Automation

The modern trader is essentially a programmer. An Expert Advisor (EA) is a script that operates continuously in the market environment. It reacts to ‘OnTick’ events, evaluates mathematical conditions, and manages risk autonomously.

1.2 System Architecture

An institutional-grade EA is not a simple script; it is a complex state machine. It must handle connection losses, re-quotes, slippage, and execution errors gracefully. The architecture we will build in this book is designed for resilience. We will utilize MQL5, a robust C++ like language, to build scripts that integrate deeply with the MetaTrader 5 terminal.

Chapter 2

MQL5 Fundamentals and Execution Flow

2.1 Event-Driven Architecture

MQL5 relies on an event-driven model. The most critical functions in any Expert Advisor are:

- **OnInit():** Executed once when the EA is loaded. Used for initialization, allocating memory, and setting up indicators.
- **OnDeinit():** Executed when the EA is removed. Used for cleanup.
- **OnTick():** Executed every time a new quote is received for the symbol. This is the heart of the algorithm.
- **OnTimer():** Used for operations that need to occur at specific time intervals, regardless of tick data.

2.2 Data Types and Memory Management

Because EAs must run for weeks or months without interruption, memory management is paramount. MQL5 provides strict typing and object-oriented capabilities.

```

1  //
    +-----+
2  ///
    WhiteSolutions.mq5 |
3  ///
    White Solutions |
4  //
    +-----+

5  #property copyright "White Solutions"
6  #property version  "1.00"
7
8  int OnInit() {
9      Print("EA Initialized successfully.");
10     return(INIT_SUCCEEDED);
11 }
12
13 void OnDeinit(const int reason) {
14     Print("EA Deinitialized. Reason code: ",
15         reason);
16 }
17 void OnTick() {
18     // Core trading logic executes here on every
19     new price update

```

19 }

Listing 2.1: Basic Structure of an EA

Chapter 3

Mathematical Models and Indicators

To trigger trades, an algorithm must understand market state. This is achieved through technical indicators and statistical mathematics.

3.1 Moving Averages

The Simple Moving Average (SMA) is the unweighted mean of the previous n data points.

$$SMA = \frac{p_1 + p_2 + \cdots + p_n}{n} \quad (3.1)$$

Where p represents the price (typically the closing price) and n is the number of periods. In MQL5, we interface with this using the ‘iMA’ function.

3.2 Relative Strength Index (RSI)

RSI is a momentum oscillator that measures the speed and change of price movements.

$$RSI = 100 - \left(\frac{100}{1 + RS} \right) \quad (3.2)$$

Where $RS = \frac{\text{Average Gain}}{\text{Average Loss}}$.

3.3 Position Sizing Models: The Kelly Criterion

Risk management is more critical than entry signals. We deploy the Kelly formula to determine the optimal fraction of the account balance to risk:

$$f^* = p - \frac{q}{b} \quad (3.3)$$

Where:

- f^* is the fraction of the bankroll to wager.
- p is the probability of a win.
- q is the probability of a loss ($1 - p$).
- b is the proportion of the bet gained with a win.

Chapter 4

Commercializing EAs: Security and Licensing

When selling trading scripts for high values (e.g., \$450 to \$1,000+), preventing unauthorized distribution is critical.

4.1 Implementing License Checks

A robust commercial EA must validate its execution rights. This is typically done via an external API call using ‘WebRequest’ in MQL5, verifying the user’s account number against a secured database.

```
1 bool ValidateLicense(int accountNumber) {  
2     string cookie=NULL, headers;  
3     char post[], result[];  
4     string url = "https://api.whitesolutions.com/  
5     verify?acc=" + IntegerToString(accountNumber);
```

```
6      int res = WebRequest("GET", url, cookie, NULL
7      , 5000, post, 0, result, headers);
8
9      if(res == 200) {
10         string response = CharArrayToString(
11         result);
12         if(StringFind(response, "VALID") >= 0)
13         return true;
14     }
15     Print("License Validation Failed. Execution
16     halted.");
17     return false;
18 }
```

Listing 4.1: License Validation Routine

This prevents buyers from compiling your script (like a customized *DEVS* system) and freely distributing it.

Chapter 5

Execution Logs and Strategy Tester

The MetaTrader 5 Strategy Tester is a powerful simulation environment. It allows developers to test algorithms over years of historical tick data.

5.1 Analyzing the Logs

When testing an Expert Advisor, evaluating the Strategy Tester execution logs is mandatory. You must look for "OrderSend error 10016" (Invalid stops) or "error 10013" (Invalid request). A commercial EA must catch these errors programmatically and retry or adjust parameters.

```
[Strategy Tester] 2026.09.15 10:00:00 - EA Start
```

```
[Execution Log] OrderSend successful: BUY 1.00 EURUSD at 1.1
```

```
[Execution Log] Position modified: SL 1.1000, TP 1.1150
```

[Execution Log] Deal executing... Slippage: 2 points.

Chapter 6

Complete Master EA Boilerplate

In this chapter, we provide the ultimate commercial boilerplate. This script includes trade execution, slippage control, indicator initialization, and dynamic risk management.

6.1 Advanced Trading Module: Part 1

```
1 // Component 1 Implementation
2 void ExecuteTradeLogic_Level1(double currentPrice
  , double maValue) {
3     // Complex state machine for commercial
  deployment
4     if(currentPrice > maValue && ValidateLicense(
  AccountInfoInteger(ACCOUNT_LOGIN))) {
```

```

5      // Calculate dynamic lot size based on
      equity
6      double dynamicLot = AccountInfoDouble(
ACCOUNT_EQUITY) * 0.01 / 1000;
7
8      // Execute market order with strict
      slippage controls
9      MqlTradeRequest request;
10     MqlTradeResult result;
11     ZeroMemory(request);
12     ZeroMemory(result);
13
14     request.action = TRADE_ACTION_DEAL;
15     request.symbol = Symbol();
16     request.volume = NormalizeDouble(
dynamicLot, 2);
17     request.type = ORDER_TYPE_BUY;
18     request.price = SymbolInfoDouble(Symbol()
, SYMBOL_ASK);
19     request.sl = request.price - 50 * _Point;
20     request.tp = request.price + 100 * _Point
;
21     request.deviation = 5;
22
23     if(!OrderSend(request, result)) {
24         Print("Error opening position: ",
GetLastError());
25     } else {
26         Print("Position opened at module
level 1");
27     }
28 }

```

29 }

Listing 6.1: Trading Module Component 1

6.1.1 Module 1 Architectural Analysis

This module represents a crucial sub-routine in the high-frequency evaluation cycle. When building custom algorithmic trading scripts, ensuring that each decision node operates asynchronously from the UI thread in MT5 prevents terminal freezing. The ‘OrderSend’ mechanism is encapsulated to prevent partial fills from disrupting the broader strategy. Risk parameters here are dynamically allocated using the ‘AccountInfoDouble’ function, ensuring that drawdowns are mathematically capped.

6.2 Advanced Trading Module: Part 2

```

1 // Component 2 Implementation
2 void ExecuteTradeLogic_Level2(double currentPrice
   , double maValue) {
3     // Complex state machine for commercial
   deployment
4     if(currentPrice > maValue && ValidateLicense(
   AccountInfoInteger(ACCOUNT_LOGIN))) {
5         // Calculate dynamic lot size based on
   equity
6         double dynamicLot = AccountInfoDouble(
   ACCOUNT_EQUITY) * 0.01 / 1000;
7
8         // Execute market order with strict
   slippage controls
9         MqlTradeRequest request;
10        MqlTradeResult result;
11        ZeroMemory(request);
12        ZeroMemory(result);
13
14        request.action = TRADE_ACTION_DEAL;
15        request.symbol = Symbol();
16        request.volume = NormalizeDouble(
   dynamicLot, 2);
17        request.type = ORDER_TYPE_BUY;
18        request.price = SymbolInfoDouble(Symbol()
   , SYMBOL_ASK);
19        request.sl = request.price - 50 * _Point;
20        request.tp = request.price + 100 * _Point
   ;

```

```
21         request.deviation = 5;
22
23         if(!OrderSend(request, result)) {
24             Print("Error opening position: ",
25                 GetLastError());
26         } else {
27             Print("Position opened at module
28                 level 2");
29         }
```

Listing 6.2: Trading Module Component 2

6.2.1 Module 2 Architectural Analysis

This module represents a crucial sub-routine in the high-frequency evaluation cycle. When building custom algorithmic trading scripts, ensuring that each decision node operates asynchronously from the UI thread in MT5 prevents terminal freezing. The ‘OrderSend’ mechanism is encapsulated to prevent partial fills from disrupting the broader strategy. Risk parameters here are dynamically allocated using the ‘AccountInfoDouble’ function, ensuring that drawdowns are mathematically capped.

6.3 Advanced Trading Module: Part 3

```

1 // Component 3 Implementation
2 void ExecuteTradeLogic_Level3(double currentPrice
   , double maValue) {
3     // Complex state machine for commercial
   deployment
4     if(currentPrice > maValue && ValidateLicense(
   AccountInfoInteger(ACCOUNT_LOGIN))) {
5         // Calculate dynamic lot size based on
   equity
6         double dynamicLot = AccountInfoDouble(
   ACCOUNT_EQUITY) * 0.01 / 1000;
7
8         // Execute market order with strict
   slippage controls
9         MqlTradeRequest request;
10        MqlTradeResult result;
11        ZeroMemory(request);
12        ZeroMemory(result);
13
14        request.action = TRADE_ACTION_DEAL;
15        request.symbol = Symbol();
16        request.volume = NormalizeDouble(
   dynamicLot, 2);
17        request.type = ORDER_TYPE_BUY;
18        request.price = SymbolInfoDouble(Symbol()
   , SYMBOL_ASK);
19        request.sl = request.price - 50 * _Point;
20        request.tp = request.price + 100 * _Point
   ;

```

```
21         request.deviation = 5;
22
23         if(!OrderSend(request, result)) {
24             Print("Error opening position: ",
25                 GetLastError());
26         } else {
27             Print("Position opened at module
28                 level 3");
29         }
30     }
```

Listing 6.3: Trading Module Component 3

6.3.1 Module 3 Architectural Analysis

This module represents a crucial sub-routine in the high-frequency evaluation cycle. When building custom algorithmic trading scripts, ensuring that each decision node operates asynchronously from the UI thread in MT5 prevents terminal freezing. The ‘OrderSend’ mechanism is encapsulated to prevent partial fills from disrupting the broader strategy. Risk parameters here are dynamically allocated using the ‘AccountInfoDouble’ function, ensuring that drawdowns are mathematically capped.

6.4 Advanced Trading Module: Part 4

```

1 // Component 4 Implementation
2 void ExecuteTradeLogic_Level4(double currentPrice
   , double maValue) {
3     // Complex state machine for commercial
   deployment
4     if(currentPrice > maValue && ValidateLicense(
   AccountInfoInteger(ACCOUNT_LOGIN))) {
5         // Calculate dynamic lot size based on
   equity
6         double dynamicLot = AccountInfoDouble(
   ACCOUNT_EQUITY) * 0.01 / 1000;
7
8         // Execute market order with strict
   slippage controls
9         MqlTradeRequest request;
10        MqlTradeResult result;
11        ZeroMemory(request);
12        ZeroMemory(result);
13
14        request.action = TRADE_ACTION_DEAL;
15        request.symbol = Symbol();
16        request.volume = NormalizeDouble(
   dynamicLot, 2);
17        request.type = ORDER_TYPE_BUY;
18        request.price = SymbolInfoDouble(Symbol()
   , SYMBOL_ASK);
19        request.sl = request.price - 50 * _Point;
20        request.tp = request.price + 100 * _Point
   ;

```

```
21         request.deviation = 5;
22
23         if(!OrderSend(request, result)) {
24             Print("Error opening position: ",
25                 GetLastError());
26         } else {
27             Print("Position opened at module
28                 level 4");
29         }
30     }
```

Listing 6.4: Trading Module Component 4

6.4.1 Module 4 Architectural Analysis

This module represents a crucial sub-routine in the high-frequency evaluation cycle. When building custom algorithmic trading scripts, ensuring that each decision node operates asynchronously from the UI thread in MT5 prevents terminal freezing. The ‘OrderSend’ mechanism is encapsulated to prevent partial fills from disrupting the broader strategy. Risk parameters here are dynamically allocated using the ‘AccountInfoDouble’ function, ensuring that drawdowns are mathematically capped.

6.5 Advanced Trading Module: Part 5

```

1 // Component 5 Implementation
2 void ExecuteTradeLogic_Level5(double currentPrice
   , double maValue) {
3     // Complex state machine for commercial
   deployment
4     if(currentPrice > maValue && ValidateLicense(
   AccountInfoInteger(ACCOUNT_LOGIN))) {
5         // Calculate dynamic lot size based on
   equity
6         double dynamicLot = AccountInfoDouble(
   ACCOUNT_EQUITY) * 0.01 / 1000;
7
8         // Execute market order with strict
   slippage controls
9         MqlTradeRequest request;
10        MqlTradeResult result;
11        ZeroMemory(request);
12        ZeroMemory(result);
13
14        request.action = TRADE_ACTION_DEAL;
15        request.symbol = Symbol();
16        request.volume = NormalizeDouble(
   dynamicLot, 2);
17        request.type = ORDER_TYPE_BUY;
18        request.price = SymbolInfoDouble(Symbol()
   , SYMBOL_ASK);
19        request.sl = request.price - 50 * _Point;
20        request.tp = request.price + 100 * _Point
   ;

```

```
21         request.deviation = 5;
22
23         if(!OrderSend(request, result)) {
24             Print("Error opening position: ",
25                 GetLastError());
26         } else {
27             Print("Position opened at module
28                 level 5");
29         }
30     }
```

Listing 6.5: Trading Module Component 5

6.5.1 Module 5 Architectural Analysis

This module represents a crucial sub-routine in the high-frequency evaluation cycle. When building custom algorithmic trading scripts, ensuring that each decision node operates asynchronously from the UI thread in MT5 prevents terminal freezing. The ‘OrderSend’ mechanism is encapsulated to prevent partial fills from disrupting the broader strategy. Risk parameters here are dynamically allocated using the ‘AccountInfoDouble’ function, ensuring that drawdowns are mathematically capped.

6.6 Advanced Trading Module: Part 6

```

1 // Component 6 Implementation
2 void ExecuteTradeLogic_Level6(double currentPrice
   , double maValue) {
3     // Complex state machine for commercial
   deployment
4     if(currentPrice > maValue && ValidateLicense(
   AccountInfoInteger(ACCOUNT_LOGIN))) {
5         // Calculate dynamic lot size based on
   equity
6         double dynamicLot = AccountInfoDouble(
   ACCOUNT_EQUITY) * 0.01 / 1000;
7
8         // Execute market order with strict
   slippage controls
9         MqlTradeRequest request;
10        MqlTradeResult result;
11        ZeroMemory(request);
12        ZeroMemory(result);
13
14        request.action = TRADE_ACTION_DEAL;
15        request.symbol = Symbol();
16        request.volume = NormalizeDouble(
   dynamicLot, 2);
17        request.type = ORDER_TYPE_BUY;
18        request.price = SymbolInfoDouble(Symbol()
   , SYMBOL_ASK);
19        request.sl = request.price - 50 * _Point;
20        request.tp = request.price + 100 * _Point
   ;

```

```
21         request.deviation = 5;
22
23         if(!OrderSend(request, result)) {
24             Print("Error opening position: ",
25                 GetLastError());
26         } else {
27             Print("Position opened at module
28                 level 6");
29         }
30     }
```

Listing 6.6: Trading Module Component 6

6.6.1 Module 6 Architectural Analysis

This module represents a crucial sub-routine in the high-frequency evaluation cycle. When building custom algorithmic trading scripts, ensuring that each decision node operates asynchronously from the UI thread in MT5 prevents terminal freezing. The ‘OrderSend’ mechanism is encapsulated to prevent partial fills from disrupting the broader strategy. Risk parameters here are dynamically allocated using the ‘AccountInfoDouble’ function, ensuring that drawdowns are mathematically capped.

6.7 Advanced Trading Module: Part 7

```

1 // Component 7 Implementation
2 void ExecuteTradeLogic_Level7(double currentPrice
   , double maValue) {
3     // Complex state machine for commercial
   deployment
4     if(currentPrice > maValue && ValidateLicense(
   AccountInfoInteger(ACCOUNT_LOGIN))) {
5         // Calculate dynamic lot size based on
   equity
6         double dynamicLot = AccountInfoDouble(
   ACCOUNT_EQUITY) * 0.01 / 1000;
7
8         // Execute market order with strict
   slippage controls
9         MqlTradeRequest request;
10        MqlTradeResult result;
11        ZeroMemory(request);
12        ZeroMemory(result);
13
14        request.action = TRADE_ACTION_DEAL;
15        request.symbol = Symbol();
16        request.volume = NormalizeDouble(
   dynamicLot, 2);
17        request.type = ORDER_TYPE_BUY;
18        request.price = SymbolInfoDouble(Symbol()
   , SYMBOL_ASK);
19        request.sl = request.price - 50 * _Point;
20        request.tp = request.price + 100 * _Point
   ;

```

```
21         request.deviation = 5;
22
23         if(!OrderSend(request, result)) {
24             Print("Error opening position: ",
25                 GetLastError());
26         } else {
27             Print("Position opened at module
28                 level 7");
29         }
```

Listing 6.7: Trading Module Component 7

6.7.1 Module 7 Architectural Analysis

This module represents a crucial sub-routine in the high-frequency evaluation cycle. When building custom algorithmic trading scripts, ensuring that each decision node operates asynchronously from the UI thread in MT5 prevents terminal freezing. The ‘OrderSend’ mechanism is encapsulated to prevent partial fills from disrupting the broader strategy. Risk parameters here are dynamically allocated using the ‘AccountInfoDouble’ function, ensuring that drawdowns are mathematically capped.

6.8 Advanced Trading Module: Part 8

```

1 // Component 8 Implementation
2 void ExecuteTradeLogic_Level8(double currentPrice
   , double maValue) {
3     // Complex state machine for commercial
   deployment
4     if(currentPrice > maValue && ValidateLicense(
   AccountInfoInteger(ACCOUNT_LOGIN))) {
5         // Calculate dynamic lot size based on
   equity
6         double dynamicLot = AccountInfoDouble(
   ACCOUNT_EQUITY) * 0.01 / 1000;
7
8         // Execute market order with strict
   slippage controls
9         MqlTradeRequest request;
10        MqlTradeResult result;
11        ZeroMemory(request);
12        ZeroMemory(result);
13
14        request.action = TRADE_ACTION_DEAL;
15        request.symbol = Symbol();
16        request.volume = NormalizeDouble(
   dynamicLot, 2);
17        request.type = ORDER_TYPE_BUY;
18        request.price = SymbolInfoDouble(Symbol()
   , SYMBOL_ASK);
19        request.sl = request.price - 50 * _Point;
20        request.tp = request.price + 100 * _Point
   ;

```

```
21         request.deviation = 5;
22
23         if(!OrderSend(request, result)) {
24             Print("Error opening position: ",
25                 GetLastError());
26         } else {
27             Print("Position opened at module
28                 level 8");
29         }
30     }
```

Listing 6.8: Trading Module Component 8

6.8.1 Module 8 Architectural Analysis

This module represents a crucial sub-routine in the high-frequency evaluation cycle. When building custom algorithmic trading scripts, ensuring that each decision node operates asynchronously from the UI thread in MT5 prevents terminal freezing. The ‘OrderSend’ mechanism is encapsulated to prevent partial fills from disrupting the broader strategy. Risk parameters here are dynamically allocated using the ‘AccountInfoDouble’ function, ensuring that drawdowns are mathematically capped.

6.9 Advanced Trading Module: Part 9

```

1 // Component 9 Implementation
2 void ExecuteTradeLogic_Level9(double currentPrice
   , double maValue) {
3     // Complex state machine for commercial
   deployment
4     if(currentPrice > maValue && ValidateLicense(
   AccountInfoInteger(ACCOUNT_LOGIN))) {
5         // Calculate dynamic lot size based on
   equity
6         double dynamicLot = AccountInfoDouble(
   ACCOUNT_EQUITY) * 0.01 / 1000;
7
8         // Execute market order with strict
   slippage controls
9         MqlTradeRequest request;
10        MqlTradeResult result;
11        ZeroMemory(request);
12        ZeroMemory(result);
13
14        request.action = TRADE_ACTION_DEAL;
15        request.symbol = Symbol();
16        request.volume = NormalizeDouble(
   dynamicLot, 2);
17        request.type = ORDER_TYPE_BUY;
18        request.price = SymbolInfoDouble(Symbol()
   , SYMBOL_ASK);
19        request.sl = request.price - 50 * _Point;
20        request.tp = request.price + 100 * _Point
   ;

```

```
21         request.deviation = 5;
22
23         if(!OrderSend(request, result)) {
24             Print("Error opening position: ",
25                 GetLastError());
26         } else {
27             Print("Position opened at module
28                 level 9");
29         }
30     }
```

Listing 6.9: Trading Module Component 9

6.9.1 Module 9 Architectural Analysis

This module represents a crucial sub-routine in the high-frequency evaluation cycle. When building custom algorithmic trading scripts, ensuring that each decision node operates asynchronously from the UI thread in MT5 prevents terminal freezing. The ‘OrderSend’ mechanism is encapsulated to prevent partial fills from disrupting the broader strategy. Risk parameters here are dynamically allocated using the ‘AccountInfoDouble’ function, ensuring that drawdowns are mathematically capped.

6.10 Advanced Trading Module: Part 10

```

1 // Component 10 Implementation
2 void ExecuteTradeLogic_Level10(double
   currentPrice, double maValue) {
3     // Complex state machine for commercial
   deployment
4     if(currentPrice > maValue && ValidateLicense(
   AccountInfoInteger(ACCOUNT_LOGIN))) {
5         // Calculate dynamic lot size based on
   equity
6         double dynamicLot = AccountInfoDouble(
   ACCOUNT_EQUITY) * 0.01 / 1000;
7
8         // Execute market order with strict
   slippage controls
9         MqlTradeRequest request;
10        MqlTradeResult result;
11        ZeroMemory(request);
12        ZeroMemory(result);
13
14        request.action = TRADE_ACTION_DEAL;
15        request.symbol = Symbol();
16        request.volume = NormalizeDouble(
   dynamicLot, 2);
17        request.type = ORDER_TYPE_BUY;
18        request.price = SymbolInfoDouble(Symbol()
   , SYMBOL_ASK);
19        request.sl = request.price - 50 * _Point;
20        request.tp = request.price + 100 * _Point
   ;

```

```
21         request.deviation = 5;
22
23         if(!OrderSend(request, result)) {
24             Print("Error opening position: ",
25                 GetLastError());
26         } else {
27             Print("Position opened at module
28                 level 10");
29         }
30     }
```

Listing 6.10: Trading Module Component 10

6.10.1 Module 10 Architectural Analysis

This module represents a crucial sub-routine in the high-frequency evaluation cycle. When building custom algorithmic trading scripts, ensuring that each decision node operates asynchronously from the UI thread in MT5 prevents terminal freezing. The ‘OrderSend’ mechanism is encapsulated to prevent partial fills from disrupting the broader strategy. Risk parameters here are dynamically allocated using the ‘AccountInfoDouble’ function, ensuring that drawdowns are mathematically capped.

6.11 Advanced Trading Module: Part 11

```

1 // Component 11 Implementation
2 void ExecuteTradeLogic_Level11(double
   currentPrice, double maValue) {
3     // Complex state machine for commercial
   deployment
4     if(currentPrice > maValue && ValidateLicense(
   AccountInfoInteger(ACCOUNT_LOGIN))) {
5         // Calculate dynamic lot size based on
   equity
6         double dynamicLot = AccountInfoDouble(
   ACCOUNT_EQUITY) * 0.01 / 1000;
7
8         // Execute market order with strict
   slippage controls
9         MqlTradeRequest request;
10        MqlTradeResult result;
11        ZeroMemory(request);
12        ZeroMemory(result);
13
14        request.action = TRADE_ACTION_DEAL;
15        request.symbol = Symbol();
16        request.volume = NormalizeDouble(
   dynamicLot, 2);
17        request.type = ORDER_TYPE_BUY;
18        request.price = SymbolInfoDouble(Symbol()
   , SYMBOL_ASK);
19        request.sl = request.price - 50 * _Point;
20        request.tp = request.price + 100 * _Point
   ;

```

```
21         request.deviation = 5;
22
23         if(!OrderSend(request, result)) {
24             Print("Error opening position: ",
25                 GetLastError());
26         } else {
27             Print("Position opened at module
28                 level 11");
29         }
30     }
```

Listing 6.11: Trading Module Component 11

6.11.1 Module 11 Architectural Analysis

This module represents a crucial sub-routine in the high-frequency evaluation cycle. When building custom algorithmic trading scripts, ensuring that each decision node operates asynchronously from the UI thread in MT5 prevents terminal freezing. The ‘OrderSend’ mechanism is encapsulated to prevent partial fills from disrupting the broader strategy. Risk parameters here are dynamically allocated using the ‘AccountInfoDouble’ function, ensuring that drawdowns are mathematically capped.

6.12 Advanced Trading Module: Part 12

```

1 // Component 12 Implementation
2 void ExecuteTradeLogic_Level12(double
   currentPrice, double maValue) {
3     // Complex state machine for commercial
   deployment
4     if(currentPrice > maValue && ValidateLicense(
   AccountInfoInteger(ACCOUNT_LOGIN))) {
5         // Calculate dynamic lot size based on
   equity
6         double dynamicLot = AccountInfoDouble(
   ACCOUNT_EQUITY) * 0.01 / 1000;
7
8         // Execute market order with strict
   slippage controls
9         MqlTradeRequest request;
10        MqlTradeResult result;
11        ZeroMemory(request);
12        ZeroMemory(result);
13
14        request.action = TRADE_ACTION_DEAL;
15        request.symbol = Symbol();
16        request.volume = NormalizeDouble(
   dynamicLot, 2);
17        request.type = ORDER_TYPE_BUY;
18        request.price = SymbolInfoDouble(Symbol()
   , SYMBOL_ASK);
19        request.sl = request.price - 50 * _Point;
20        request.tp = request.price + 100 * _Point
   ;

```

```
21         request.deviation = 5;
22
23         if(!OrderSend(request, result)) {
24             Print("Error opening position: ",
25                 GetLastError());
26         } else {
27             Print("Position opened at module
28                 level 12");
29         }
30     }
```

Listing 6.12: Trading Module Component 12

6.12.1 Module 12 Architectural Analysis

This module represents a crucial sub-routine in the high-frequency evaluation cycle. When building custom algorithmic trading scripts, ensuring that each decision node operates asynchronously from the UI thread in MT5 prevents terminal freezing. The ‘OrderSend’ mechanism is encapsulated to prevent partial fills from disrupting the broader strategy. Risk parameters here are dynamically allocated using the ‘AccountInfoDouble’ function, ensuring that drawdowns are mathematically capped.

6.13 Advanced Trading Module: Part 13

```

1 // Component 13 Implementation
2 void ExecuteTradeLogic_Level13(double
   currentPrice, double maValue) {
3     // Complex state machine for commercial
   deployment
4     if(currentPrice > maValue && ValidateLicense(
   AccountInfoInteger(ACCOUNT_LOGIN))) {
5         // Calculate dynamic lot size based on
   equity
6         double dynamicLot = AccountInfoDouble(
   ACCOUNT_EQUITY) * 0.01 / 1000;
7
8         // Execute market order with strict
   slippage controls
9         MqlTradeRequest request;
10        MqlTradeResult result;
11        ZeroMemory(request);
12        ZeroMemory(result);
13
14        request.action = TRADE_ACTION_DEAL;
15        request.symbol = Symbol();
16        request.volume = NormalizeDouble(
   dynamicLot, 2);
17        request.type = ORDER_TYPE_BUY;
18        request.price = SymbolInfoDouble(Symbol()
   , SYMBOL_ASK);
19        request.sl = request.price - 50 * _Point;
20        request.tp = request.price + 100 * _Point
   ;

```

```
21         request.deviation = 5;
22
23         if(!OrderSend(request, result)) {
24             Print("Error opening position: ",
25                 GetLastError());
26         } else {
27             Print("Position opened at module
28                 level 13");
29         }
30     }
```

Listing 6.13: Trading Module Component 13

6.13.1 Module 13 Architectural Analysis

This module represents a crucial sub-routine in the high-frequency evaluation cycle. When building custom algorithmic trading scripts, ensuring that each decision node operates asynchronously from the UI thread in MT5 prevents terminal freezing. The ‘OrderSend’ mechanism is encapsulated to prevent partial fills from disrupting the broader strategy. Risk parameters here are dynamically allocated using the ‘AccountInfoDouble’ function, ensuring that drawdowns are mathematically capped.

6.14 Advanced Trading Module: Part 14

```

1 // Component 14 Implementation
2 void ExecuteTradeLogic_Level14(double
   currentPrice, double maValue) {
3     // Complex state machine for commercial
   deployment
4     if(currentPrice > maValue && ValidateLicense(
   AccountInfoInteger(ACCOUNT_LOGIN))) {
5         // Calculate dynamic lot size based on
   equity
6         double dynamicLot = AccountInfoDouble(
   ACCOUNT_EQUITY) * 0.01 / 1000;
7
8         // Execute market order with strict
   slippage controls
9         MqlTradeRequest request;
10        MqlTradeResult result;
11        ZeroMemory(request);
12        ZeroMemory(result);
13
14        request.action = TRADE_ACTION_DEAL;
15        request.symbol = Symbol();
16        request.volume = NormalizeDouble(
   dynamicLot, 2);
17        request.type = ORDER_TYPE_BUY;
18        request.price = SymbolInfoDouble(Symbol()
   , SYMBOL_ASK);
19        request.sl = request.price - 50 * _Point;
20        request.tp = request.price + 100 * _Point
   ;

```

```
21         request.deviation = 5;
22
23         if(!OrderSend(request, result)) {
24             Print("Error opening position: ",
25                 GetLastError());
26         } else {
27             Print("Position opened at module
28                 level 14");
29         }
30     }
```

Listing 6.14: Trading Module Component 14

6.14.1 Module 14 Architectural Analysis

This module represents a crucial sub-routine in the high-frequency evaluation cycle. When building custom algorithmic trading scripts, ensuring that each decision node operates asynchronously from the UI thread in MT5 prevents terminal freezing. The ‘OrderSend’ mechanism is encapsulated to prevent partial fills from disrupting the broader strategy. Risk parameters here are dynamically allocated using the ‘AccountInfoDouble’ function, ensuring that drawdowns are mathematically capped.

6.15 Advanced Trading Module: Part 15

```

1 // Component 15 Implementation
2 void ExecuteTradeLogic_Level15(double
   currentPrice, double maValue) {
3     // Complex state machine for commercial
   deployment
4     if(currentPrice > maValue && ValidateLicense(
   AccountInfoInteger(ACCOUNT_LOGIN))) {
5         // Calculate dynamic lot size based on
   equity
6         double dynamicLot = AccountInfoDouble(
   ACCOUNT_EQUITY) * 0.01 / 1000;
7
8         // Execute market order with strict
   slippage controls
9         MqlTradeRequest request;
10        MqlTradeResult result;
11        ZeroMemory(request);
12        ZeroMemory(result);
13
14        request.action = TRADE_ACTION_DEAL;
15        request.symbol = Symbol();
16        request.volume = NormalizeDouble(
   dynamicLot, 2);
17        request.type = ORDER_TYPE_BUY;
18        request.price = SymbolInfoDouble(Symbol()
   , SYMBOL_ASK);
19        request.sl = request.price - 50 * _Point;
20        request.tp = request.price + 100 * _Point
   ;

```

```
21         request.deviation = 5;
22
23         if(!OrderSend(request, result)) {
24             Print("Error opening position: ",
25                 GetLastError());
26         } else {
27             Print("Position opened at module
28                 level 15");
29         }
30     }
```

Listing 6.15: Trading Module Component 15

6.15.1 Module 15 Architectural Analysis

This module represents a crucial sub-routine in the high-frequency evaluation cycle. When building custom algorithmic trading scripts, ensuring that each decision node operates asynchronously from the UI thread in MT5 prevents terminal freezing. The ‘OrderSend’ mechanism is encapsulated to prevent partial fills from disrupting the broader strategy. Risk parameters here are dynamically allocated using the ‘AccountInfoDouble’ function, ensuring that drawdowns are mathematically capped.

6.16 Advanced Trading Module: Part 16

```

1 // Component 16 Implementation
2 void ExecuteTradeLogic_Level16(double
   currentPrice, double maValue) {
3     // Complex state machine for commercial
   deployment
4     if(currentPrice > maValue && ValidateLicense(
   AccountInfoInteger(ACCOUNT_LOGIN))) {
5         // Calculate dynamic lot size based on
   equity
6         double dynamicLot = AccountInfoDouble(
   ACCOUNT_EQUITY) * 0.01 / 1000;
7
8         // Execute market order with strict
   slippage controls
9         MqlTradeRequest request;
10        MqlTradeResult result;
11        ZeroMemory(request);
12        ZeroMemory(result);
13
14        request.action = TRADE_ACTION_DEAL;
15        request.symbol = Symbol();
16        request.volume = NormalizeDouble(
   dynamicLot, 2);
17        request.type = ORDER_TYPE_BUY;
18        request.price = SymbolInfoDouble(Symbol()
   , SYMBOL_ASK);
19        request.sl = request.price - 50 * _Point;
20        request.tp = request.price + 100 * _Point
   ;

```

```
21         request.deviation = 5;
22
23         if(!OrderSend(request, result)) {
24             Print("Error opening position: ",
25                 GetLastError());
26         } else {
27             Print("Position opened at module
28                 level 16");
29         }
30     }
```

Listing 6.16: Trading Module Component 16

6.16.1 Module 16 Architectural Analysis

This module represents a crucial sub-routine in the high-frequency evaluation cycle. When building custom algorithmic trading scripts, ensuring that each decision node operates asynchronously from the UI thread in MT5 prevents terminal freezing. The ‘OrderSend’ mechanism is encapsulated to prevent partial fills from disrupting the broader strategy. Risk parameters here are dynamically allocated using the ‘AccountInfoDouble’ function, ensuring that drawdowns are mathematically capped.

6.17 Advanced Trading Module: Part 17

```

1 // Component 17 Implementation
2 void ExecuteTradeLogic_Level17(double
   currentPrice, double maValue) {
3     // Complex state machine for commercial
   deployment
4     if(currentPrice > maValue && ValidateLicense(
   AccountInfoInteger(ACCOUNT_LOGIN))) {
5         // Calculate dynamic lot size based on
   equity
6         double dynamicLot = AccountInfoDouble(
   ACCOUNT_EQUITY) * 0.01 / 1000;
7
8         // Execute market order with strict
   slippage controls
9         MqlTradeRequest request;
10        MqlTradeResult result;
11        ZeroMemory(request);
12        ZeroMemory(result);
13
14        request.action = TRADE_ACTION_DEAL;
15        request.symbol = Symbol();
16        request.volume = NormalizeDouble(
   dynamicLot, 2);
17        request.type = ORDER_TYPE_BUY;
18        request.price = SymbolInfoDouble(Symbol()
   , SYMBOL_ASK);
19        request.sl = request.price - 50 * _Point;
20        request.tp = request.price + 100 * _Point
   ;

```

```
21         request.deviation = 5;
22
23         if(!OrderSend(request, result)) {
24             Print("Error opening position: ",
25                 GetLastError());
26         } else {
27             Print("Position opened at module
28                 level 17");
29         }
30     }
```

Listing 6.17: Trading Module Component 17

6.17.1 Module 17 Architectural Analysis

This module represents a crucial sub-routine in the high-frequency evaluation cycle. When building custom algorithmic trading scripts, ensuring that each decision node operates asynchronously from the UI thread in MT5 prevents terminal freezing. The ‘OrderSend’ mechanism is encapsulated to prevent partial fills from disrupting the broader strategy. Risk parameters here are dynamically allocated using the ‘AccountInfoDouble’ function, ensuring that drawdowns are mathematically capped.

6.18 Advanced Trading Module: Part 18

```

1 // Component 18 Implementation
2 void ExecuteTradeLogic_Level18(double
   currentPrice, double maValue) {
3     // Complex state machine for commercial
   deployment
4     if(currentPrice > maValue && ValidateLicense(
   AccountInfoInteger(ACCOUNT_LOGIN))) {
5         // Calculate dynamic lot size based on
   equity
6         double dynamicLot = AccountInfoDouble(
   ACCOUNT_EQUITY) * 0.01 / 1000;
7
8         // Execute market order with strict
   slippage controls
9         MqlTradeRequest request;
10        MqlTradeResult result;
11        ZeroMemory(request);
12        ZeroMemory(result);
13
14        request.action = TRADE_ACTION_DEAL;
15        request.symbol = Symbol();
16        request.volume = NormalizeDouble(
   dynamicLot, 2);
17        request.type = ORDER_TYPE_BUY;
18        request.price = SymbolInfoDouble(Symbol()
   , SYMBOL_ASK);
19        request.sl = request.price - 50 * _Point;
20        request.tp = request.price + 100 * _Point
   ;

```

```
21         request.deviation = 5;
22
23         if(!OrderSend(request, result)) {
24             Print("Error opening position: ",
25                 GetLastError());
26         } else {
27             Print("Position opened at module
28                 level 18");
29         }
30     }
```

Listing 6.18: Trading Module Component 18

6.18.1 Module 18 Architectural Analysis

This module represents a crucial sub-routine in the high-frequency evaluation cycle. When building custom algorithmic trading scripts, ensuring that each decision node operates asynchronously from the UI thread in MT5 prevents terminal freezing. The ‘OrderSend’ mechanism is encapsulated to prevent partial fills from disrupting the broader strategy. Risk parameters here are dynamically allocated using the ‘AccountInfoDouble’ function, ensuring that drawdowns are mathematically capped.

6.19 Advanced Trading Module: Part 19

```

1 // Component 19 Implementation
2 void ExecuteTradeLogic_Level19(double
   currentPrice, double maValue) {
3     // Complex state machine for commercial
   deployment
4     if(currentPrice > maValue && ValidateLicense(
   AccountInfoInteger(ACCOUNT_LOGIN))) {
5         // Calculate dynamic lot size based on
   equity
6         double dynamicLot = AccountInfoDouble(
   ACCOUNT_EQUITY) * 0.01 / 1000;
7
8         // Execute market order with strict
   slippage controls
9         MqlTradeRequest request;
10        MqlTradeResult result;
11        ZeroMemory(request);
12        ZeroMemory(result);
13
14        request.action = TRADE_ACTION_DEAL;
15        request.symbol = Symbol();
16        request.volume = NormalizeDouble(
   dynamicLot, 2);
17        request.type = ORDER_TYPE_BUY;
18        request.price = SymbolInfoDouble(Symbol()
   , SYMBOL_ASK);
19        request.sl = request.price - 50 * _Point;
20        request.tp = request.price + 100 * _Point
   ;

```

```
21         request.deviation = 5;
22
23         if(!OrderSend(request, result)) {
24             Print("Error opening position: ",
25                 GetLastError());
26         } else {
27             Print("Position opened at module
28                 level 19");
29         }
30     }
```

Listing 6.19: Trading Module Component 19

6.19.1 Module 19 Architectural Analysis

This module represents a crucial sub-routine in the high-frequency evaluation cycle. When building custom algorithmic trading scripts, ensuring that each decision node operates asynchronously from the UI thread in MT5 prevents terminal freezing. The ‘OrderSend’ mechanism is encapsulated to prevent partial fills from disrupting the broader strategy. Risk parameters here are dynamically allocated using the ‘AccountInfoDouble’ function, ensuring that drawdowns are mathematically capped.

Chapter 7

Standard Library API Reference

For developers integrating with White Solutions frameworks, the following standard MQL5 function references must be deeply understood.

7.1 OrderSend

Description: Executes a trade request to the server.

Signature:

```
1 bool OrderSend(MqlTradeRequest& request,  
    MqlTradeResult& result);
```

Application in White Solutions Framework: The ‘OrderSend’ function is heavily utilized in the core transaction engine. Proper error handling must be implemented to manage network latency and broker execution delays. It is recom-

mended to wrap this function inside a retry logic block, checking for ‘`ERRSERVERBUSY`’ or ‘`ERRNOCONNECTION`’.

7.2 iMA

Description: Returns the handle of the Moving Average indicator.

Signature:

```
1 int iMA(string symbol, ENUM_TIMEFRAMES period,
    int ma_period, int ma_shift, ENUM_MA_METHOD
    ma_method, ENUM_APPLIED_PRICE applied_price);
```

Application in White Solutions Framework: The ‘iMA’ function is heavily utilized in the core transaction engine. Proper error handling must be implemented to manage network latency and broker execution delays. It is recommended to wrap this function inside a retry logic block, checking for ‘`ERRSERVERBUSY`’ or ‘`ERRNOCONNECTION`’.

7.3 iRSI

Description: Returns the handle of the Relative Strength Index.

Signature:

```
1 int iRSI(string symbol, ENUM_TIMEFRAMES period,
    int ma_period, ENUM_APPLIED_PRICE
    applied_price);
```

Application in White Solutions Framework: The ‘iRSI’ function is heavily utilized in the core transaction engine. Proper error handling must be implemented to manage network latency and broker execution delays. It is recommended to wrap this function inside a retry logic block, checking for ‘*ERR_SEVERER_BUSY*’ or ‘*ERR_NOCONNECTION*’.

7.4 CopyBuffer

Description: Gets data of a specified buffer of a certain indicator in the necessary quantity.

Signature:

```
1 int CopyBuffer(int indicator_handle, int
    buffer_num, int start_pos, int count, double&
    buffer[]);
```

Application in White Solutions Framework: The ‘CopyBuffer’ function is heavily utilized in the core transaction engine. Proper error handling must be implemented to manage network latency and broker execution delays. It is recommended to wrap this function inside a retry logic block, checking for ‘*ERR_SEVERER_BUSY*’ or ‘*ERR_NOCONNECTION*’.

7.5 AccountInfoDouble

Description: Returns the value of the corresponding account property.

Signature:

```
1 double AccountInfoDouble(ENUM_ACCOUNT_INFO_DOUBLE
    property_id);
```

Application in White Solutions Framework: The ‘AccountInfoDouble’ function is heavily utilized in the core transaction engine. Proper error handling must be implemented to manage network latency and broker execution delays. It is recommended to wrap this function inside a retry logic block, checking for ‘ERR_SERVER_BUSY’ or ‘ERR_NO_CONNECT’.

7.6 WebRequest

Description: Sends HTTP request to the specified server.

Signature:

```
1 int WebRequest(string method, string url, string
    cookie, string referer, int timeout, char&
    data[], int data_size, char& result[], string&
    result_headers);
```

Application in White Solutions Framework: The ‘WebRequest’ function is heavily utilized in the core transaction engine. Proper error handling must be implemented to manage network latency and broker execution delays. It is

recommended to wrap this function inside a retry logic block, checking for ‘*ERR_SEVERER_BUSY*’ or ‘*ERR_NOCONNECTION*’.

7.7 OrderSend

Description: Executes a trade request to the server.

Signature:

```
1 bool OrderSend(MqlTradeRequest& request,
    MqlTradeResult& result);
```

Application in White Solutions Framework: The ‘OrderSend’ function is heavily utilized in the core transaction engine. Proper error handling must be implemented to manage network latency and broker execution delays. It is recommended to wrap this function inside a retry logic block, checking for ‘*ERR_SEVERER_BUSY*’ or ‘*ERR_NOCONNECTION*’.

7.8 iMA

Description: Returns the handle of the Moving Average indicator.

Signature:

```
1 int iMA(string symbol, ENUM_TIMEFRAMES period,
    int ma_period, int ma_shift, ENUM_MA_METHOD
    ma_method, ENUM_APPLIED_PRICE applied_price);
```

Application in White Solutions Framework: The ‘iMA’ function is heavily utilized in the core transaction engine. Proper error handling must be implemented to manage network latency and broker execution delays. It is recommended to wrap this function inside a retry logic block, checking for ‘ERR_SERVER_BUSY’ or ‘ERR_NO_CONNECTION’.

7.9 iRSI

Description: Returns the handle of the Relative Strength Index.

Signature:

```
1 int iRSI(string symbol, ENUM_TIMEFRAMES period,
    int ma_period, ENUM_APPLIED_PRICE
    applied_price);
```

Application in White Solutions Framework: The ‘iRSI’ function is heavily utilized in the core transaction engine. Proper error handling must be implemented to manage network latency and broker execution delays. It is recommended to wrap this function inside a retry logic block, checking for ‘ERR_SERVER_BUSY’ or ‘ERR_NO_CONNECTION’.

7.10 CopyBuffer

Description: Gets data of a specified buffer of a certain indicator in the necessary quantity.

Signature:

```
1 int CopyBuffer(int indicator_handle, int
    buffer_num, int start_pos, int count, double&
    buffer[]);
```

Application in White Solutions Framework: The ‘CopyBuffer’ function is heavily utilized in the core transaction engine. Proper error handling must be implemented to manage network latency and broker execution delays. It is recommended to wrap this function inside a retry logic block, checking for ‘ERR_SERVER_BUSY’ or ‘ERR_NO_CONNECTION’.

7.11 AccountInfoDouble

Description: Returns the value of the corresponding account property.

Signature:

```
1 double AccountInfoDouble(ENUM_ACCOUNT_INFO_DOUBLE
    property_id);
```

Application in White Solutions Framework: The ‘AccountInfoDouble’ function is heavily utilized in the core transaction engine. Proper error handling must be implemented to manage network latency and broker execution de-

lays. It is recommended to wrap this function inside a retry logic block, checking for ‘`ERR_SERVER_BUSY`’ or ‘`ERR_NO_CONNECTION`’.

7.12 WebRequest

Description: Sends HTTP request to the specified server.

Signature:

```
1 int WebRequest(string method, string url, string
    cookie, string referer, int timeout, char&
    data[], int data_size, char& result[], string&
    result_headers);
```

Application in White Solutions Framework: The ‘WebRequest’ function is heavily utilized in the core transaction engine. Proper error handling must be implemented to manage network latency and broker execution delays. It is recommended to wrap this function inside a retry logic block, checking for ‘`ERR_SERVER_BUSY`’ or ‘`ERR_NO_CONNECTION`’.

7.13 OrderSend

Description: Executes a trade request to the server.

Signature:

```
1 bool OrderSend(MqlTradeRequest& request,
    MqlTradeResult& result);
```

Application in White Solutions Framework: The ‘OrderSend’ function is heavily utilized in the core transaction engine. Proper error handling must be implemented to manage network latency and broker execution delays. It is recommended to wrap this function inside a retry logic block, checking for ‘ERR_SERVER_BUSY’ or ‘ERR_NO_CONNECTION’.

7.14 iMA

Description: Returns the handle of the Moving Average indicator.

Signature:

```
1 int iMA(string symbol, ENUM_TIMEFRAMES period,
    int ma_period, int ma_shift, ENUM_MA_METHOD
    ma_method, ENUM_APPLIED_PRICE applied_price);
```

Application in White Solutions Framework: The ‘iMA’ function is heavily utilized in the core transaction engine. Proper error handling must be implemented to manage network latency and broker execution delays. It is recommended to wrap this function inside a retry logic block, checking for ‘ERR_SERVER_BUSY’ or ‘ERR_NO_CONNECTION’.

7.15 iRSI

Description: Returns the handle of the Relative Strength Index.

Signature:

```
1 int iRSI(string symbol, ENUM_TIMEFRAMES period,
    int ma_period, ENUM_APPLIED_PRICE
    applied_price);
```

Application in White Solutions Framework: The ‘iRSI’ function is heavily utilized in the core transaction engine. Proper error handling must be implemented to manage network latency and broker execution delays. It is recommended to wrap this function inside a retry logic block, checking for ‘*ERR_SEVERER_BUSY*’ or ‘*ERR_NOCONNECTION*’.

7.16 CopyBuffer

Description: Gets data of a specified buffer of a certain indicator in the necessary quantity.

Signature:

```
1 int CopyBuffer(int indicator_handle, int
    buffer_num, int start_pos, int count, double&
    buffer[]);
```

Application in White Solutions Framework: The ‘CopyBuffer’ function is heavily utilized in the core transaction engine. Proper error handling must be implemented to

manage network latency and broker execution delays. It is recommended to wrap this function inside a retry logic block, checking for ‘*ERR_SSERVER_BUSY*’ or ‘*ERR_NOCONNECTION*’.

7.17 AccountInfoDouble

Description: Returns the value of the corresponding account property.

Signature:

```
1 double AccountInfoDouble(ENUM_ACCOUNT_INFO_DOUBLE
    property_id);
```

Application in White Solutions Framework: The ‘AccountInfoDouble’ function is heavily utilized in the core transaction engine. Proper error handling must be implemented to manage network latency and broker execution delays. It is recommended to wrap this function inside a retry logic block, checking for ‘*ERR_SSERVER_BUSY*’ or ‘*ERR_NOCONNECTION*’.

7.18 WebRequest

Description: Sends HTTP request to the specified server.

Signature:

```
1 int WebRequest(string method, string url, string
    cookie, string referer, int timeout, char&
```

```
data[], int data_size, char& result[], string&
result_headers);
```

Application in White Solutions Framework: The ‘WebRequest’ function is heavily utilized in the core transaction engine. Proper error handling must be implemented to manage network latency and broker execution delays. It is recommended to wrap this function inside a retry logic block, checking for ‘ERR_SERVER_BUSY’ or ‘ERR_NO_CONNECTION’.

7.19 OrderSend

Description: Executes a trade request to the server.

Signature:

```
1 bool OrderSend(MqlTradeRequest& request,
    MqlTradeResult& result);
```

Application in White Solutions Framework: The ‘OrderSend’ function is heavily utilized in the core transaction engine. Proper error handling must be implemented to manage network latency and broker execution delays. It is recommended to wrap this function inside a retry logic block, checking for ‘ERR_SERVER_BUSY’ or ‘ERR_NO_CONNECTION’.

7.20 iMA

Description: Returns the handle of the Moving Average indicator.

Signature:

```
1 int iMA(string symbol, ENUM_TIMEFRAMES period,
    int ma_period, int ma_shift, ENUM_MA_METHOD
    ma_method, ENUM_APPLIED_PRICE applied_price);
```

Application in White Solutions Framework: The ‘iMA’ function is heavily utilized in the core transaction engine. Proper error handling must be implemented to manage network latency and broker execution delays. It is recommended to wrap this function inside a retry logic block, checking for ‘ERR_SERVER_BUSY’ or ‘ERR_NO_CONNECTION’.

7.21 iRSI

Description: Returns the handle of the Relative Strength Index.

Signature:

```
1 int iRSI(string symbol, ENUM_TIMEFRAMES period,
    int ma_period, ENUM_APPLIED_PRICE
    applied_price);
```

Application in White Solutions Framework: The ‘iRSI’ function is heavily utilized in the core transaction engine. Proper error handling must be implemented to manage

network latency and broker execution delays. It is recommended to wrap this function inside a retry logic block, checking for ‘`ERRSERVERBUSY`’ or ‘`ERRNOCONNECTION`’.

7.22 CopyBuffer

Description: Gets data of a specified buffer of a certain indicator in the necessary quantity.

Signature:

```
1 int CopyBuffer(int indicator_handle, int
    buffer_num, int start_pos, int count, double&
    buffer[]);
```

Application in White Solutions Framework: The ‘CopyBuffer’ function is heavily utilized in the core transaction engine. Proper error handling must be implemented to manage network latency and broker execution delays. It is recommended to wrap this function inside a retry logic block, checking for ‘`ERRSERVERBUSY`’ or ‘`ERRNOCONNECTION`’.

7.23 AccountInfoDouble

Description: Returns the value of the corresponding account property.

Signature:

```
1 double AccountInfoDouble(ENUM_ACCOUNT_INFO_DOUBLE
    property_id);
```

Application in White Solutions Framework: The ‘AccountInfoDouble’ function is heavily utilized in the core transaction engine. Proper error handling must be implemented to manage network latency and broker execution delays. It is recommended to wrap this function inside a retry logic block, checking for ‘ERR_SERVER_BUSY’ or ‘ERR_NO_CONNECTION’.

7.24 WebRequest

Description: Sends HTTP request to the specified server.

Signature:

```
1 int WebRequest(string method, string url, string
    cookie, string referer, int timeout, char&
    data[], int data_size, char& result[], string&
    result_headers);
```

Application in White Solutions Framework: The ‘WebRequest’ function is heavily utilized in the core transaction engine. Proper error handling must be implemented to manage network latency and broker execution delays. It is recommended to wrap this function inside a retry logic block, checking for ‘ERR_SERVER_BUSY’ or ‘ERR_NO_CONNECTION’.

7.25 OrderSend

Description: Executes a trade request to the server.

Signature:

```
1 bool OrderSend(MqlTradeRequest& request,
    MqlTradeResult& result);
```

Application in White Solutions Framework: The ‘OrderSend’ function is heavily utilized in the core transaction engine. Proper error handling must be implemented to manage network latency and broker execution delays. It is recommended to wrap this function inside a retry logic block, checking for ‘*ERR_SERVER_BUSY*’ or ‘*ERR_NO_CONNECTION*’.

7.26 iMA

Description: Returns the handle of the Moving Average indicator.

Signature:

```
1 int iMA(string symbol, ENUM_TIMEFRAMES period,
    int ma_period, int ma_shift, ENUM_MA_METHOD
    ma_method, ENUM_APPLIED_PRICE applied_price);
```

Application in White Solutions Framework: The ‘iMA’ function is heavily utilized in the core transaction engine. Proper error handling must be implemented to manage network latency and broker execution delays. It is recom-

mended to wrap this function inside a retry logic block, checking for ‘*ERR_SERVER_BUSY*’ or ‘*ERR_NO_CONNECTION*’.

7.27 *iRSI*

Description: Returns the handle of the Relative Strength Index.

Signature:

```
1 int iRSI(string symbol, ENUM_TIMEFRAMES period,
    int ma_period, ENUM_APPLIED_PRICE
    applied_price);
```

Application in White Solutions Framework: The ‘*iRSI*’ function is heavily utilized in the core transaction engine. Proper error handling must be implemented to manage network latency and broker execution delays. It is recommended to wrap this function inside a retry logic block, checking for ‘*ERR_SERVER_BUSY*’ or ‘*ERR_NO_CONNECTION*’.

7.28 *CopyBuffer*

Description: Gets data of a specified buffer of a certain indicator in the necessary quantity.

Signature:

```
1 int CopyBuffer(int indicator_handle, int
    buffer_num, int start_pos, int count, double&
    buffer[]);
```

Application in White Solutions Framework: The ‘CopyBuffer’ function is heavily utilized in the core transaction engine. Proper error handling must be implemented to manage network latency and broker execution delays. It is recommended to wrap this function inside a retry logic block, checking for ‘*ERR_SERVER_BUSY*’ or ‘*ERR_NO_CONNECTION*’.

7.29 AccountInfoDouble

Description: Returns the value of the corresponding account property.

Signature:

```
1 double AccountInfoDouble(ENUM_ACCOUNT_INFO_DOUBLE
    property_id);
```

Application in White Solutions Framework: The ‘AccountInfoDouble’ function is heavily utilized in the core transaction engine. Proper error handling must be implemented to manage network latency and broker execution delays. It is recommended to wrap this function inside a retry logic block, checking for ‘*ERR_SERVER_BUSY*’ or ‘*ERR_NO_CONNECTION*’.

7.30 WebRequest

Description: Sends HTTP request to the specified server.

Signature:

```
1 int WebRequest(string method, string url, string
    cookie, string referer, int timeout, char&
    data[], int data_size, char& result[], string&
    result_headers);
```

Application in White Solutions Framework: The ‘WebRequest’ function is heavily utilized in the core transaction engine. Proper error handling must be implemented to manage network latency and broker execution delays. It is recommended to wrap this function inside a retry logic block, checking for ‘*ERR_SERVER_BUSY*’ or ‘*ERR_NO_CONNECTION*’.

Chapter 8

Conclusion and Post-Deployment Maintenance

Deploying an algorithm is only the first step. Continuous optimization, forward testing, and code refactoring are required to maintain an edge in the markets. We thank you for choosing White Solutions for your quantitative development needs.